

Final Exam Study Notes

Course: CIS106 Linux Fundamentals

Topic: GitHub, Markdown, Linux Commands, Paths, Redirection, Wildcards, Brace Expansion, and Shell Scripting

4. Questions

a. How to Clone a GitHub Repository

Cloning a GitHub repository means downloading a copy of a repository from GitHub to your computer or virtual machine.

Basic Syntax

```
git clone repository_url
```

Example

```
git clone https://github.com/username/repository-name.git
```

Steps

1. Go to **GitHub**.
2. Open the repository.
3. Click the green **Code** button.
4. Copy the repository URL.
5. Open the terminal in Debian.
6. Run the `git clone` command.
7. Open the folder in VS Code.

Example Workflow

```
cd Documents
git clone https://github.com/username/cis106.git
cd cis106
```

Explanation

- `cd Documents` moves into the Documents directory.
- `git clone` downloads the GitHub repository.
- `cd cis106` moves into the cloned repository.

b. How to Use Git Commands

Git commands are used to manage changes in files and upload work to GitHub.

Common Git Commands

Command	Meaning
<code>git clone</code>	Downloads a repository from GitHub
<code>git status</code>	Shows the status of files
<code>git pull</code>	Downloads the newest changes from GitHub
<code>git add .</code>	Adds all changed files to staging
<code>git commit -m "message"</code>	Saves the changes with a message
<code>git push</code>	Uploads changes to GitHub

Common Git Workflow

```
git pull
git status
git add .
git commit -m "Updated final exam notes"
git push
```

Explanation

- `git pull` gets the latest changes.
- `git status` checks what files changed.
- `git add .` adds all files.
- `git commit -m` saves the changes with a message.
- `git push` uploads the changes to GitHub.

Example: Uploading a Markdown File to GitHub

```
git status
git add finalNotes.md
git commit -m "Added final exam study notes"
git push
```

c. How to Write a Markdown File That Contains Images and Proper Formatting

Markdown is used to write formatted notes using simple symbols.

Markdown files end with:

```
.md
```

Example:

```
finalNotes.md
```

Headings

Use `#` symbols to create headings.

```
# Heading 1
## Heading 2
### Heading 3
```

Bold Text

```
**This text is bold**
```

Example:

```
**Important command**
```

Italic Text

```
*This text is italic*
```

Example:

```
*This is a note*
```

Inline Code

Use backticks for short commands inside a sentence.

```
Use the `ls` command to list files.
```

Code Blocks

Use three backticks before and after commands.

```
```bash
ls
pwd
cd Documents
```
```

Bullet List

- First item
 - Second item
 - Third item
-

Numbered List

1. Step one
 2. Step two
 3. Step three
-

Tables

```
Command	Meaning
```

```
| `ls` | Lists files |  
| `pwd` | Shows current directory |
```

Images in Markdown

To insert an image, use this format:

```
![Image description](image-name.png)
```

Example

```
![GitHub Screenshot](github-screenshot.png)
```

Image Inside a Folder

If the image is inside a folder named `images`, write:

```
![Terminal Screenshot](images/terminal.png)
```

d. How to Convert a Markdown File to PDF

The CIS106 final exam guide says you should know how to convert a Markdown file to PDF.

One way to do this is with VS Code and a Markdown PDF extension. Another way is using `pandoc`.

Method 1: Using VS Code

1. Open the Markdown file in VS Code.
 2. Make sure a Markdown PDF extension is installed.
 3. Right-click the Markdown file.
 4. Choose the option to export or convert to PDF.
-

Method 2: Using Pandoc

Install Pandoc:

```
sudo apt install pandoc
```

Convert the Markdown file to PDF:

```
pandoc finalNotes.md -o finalNotes.pdf
```

Explanation

| Part | Meaning |
|-----------------------------|-------------------------------|
| <code>pandoc</code> | Program used to convert files |
| <code>finalNotes.md</code> | Input Markdown file |
| <code>-o</code> | Output option |
| <code>finalNotes.pdf</code> | Output PDF file |

5. How to Compress Zip a Directory or Folder in Debian

To compress a directory in Debian, use the `zip` command.

Install Zip

```
sudo apt install zip
```

Zip a Folder

```
zip -r final_exam_study_notes.zip final_exam_study_notes
```

Explanation

| Part | Meaning |
|---|---------------------------------------|
| <code>zip</code> | Creates a zip file |
| <code>-r</code> | Includes everything inside the folder |
| <code>final_exam_study_notes.zip</code> | Name of the zip file |
| <code>final_exam_study_notes</code> | Folder being compressed |

Example

```
zip -r lab_files.zip lab_files
```

This creates a compressed file named `lab_files.zip`.

6. Absolute Paths and Relative Paths

The Linux filesystem uses paths to locate files and directories.

Absolute Path

An **absolute path** starts from the root directory `/`.

It gives the complete location of a file or folder.

Example

```
/home/student/Documents/finalNotes.md
```

Creating a File Using an Absolute Path

```
touch /home/student/Documents/finalNotes.md
```

Moving to a Directory Using an Absolute Path

```
cd /home/student/Documents
```

Listing Files Using an Absolute Path

```
ls /home/student/Documents
```

Relative Path

A **relative path** starts from the current directory.

It does not start with `/`.

Example

```
Documents/finalNotes.md
```

Creating a File Using a Relative Path

```
touch finalNotes.md
```

Creating a File Inside a Folder Using a Relative Path

```
touch notes/finalNotes.md
```

Moving Using a Relative Path

```
cd Documents
```

When to Use Each Path

| Path Type | When to Use It |
|---------------|--|
| Absolute path | When you want to give the full location |
| Relative path | When you are already near the file or folder |

7. How to Work With Manual Pages Man Command

The `man` command opens the manual page for a command.

Manual pages explain what a command does, its syntax, and options.

Syntax

```
man command
```

Example


```
man ls
```

This opens the manual page for the `ls` command.

Useful Keys in Manual Pages

| Key | What It Does |
|--------------------|--------------------------------|
| <code>Space</code> | Moves down one page |
| <code>b</code> | Moves back one page |
| <code>/word</code> | Searches for a word |
| <code>n</code> | Goes to the next search result |
| <code>q</code> | Quits the manual page |

8. How to Parse Search for Specific Words in the Manual Page

To search inside a manual page, first open the manual page.

Example

```
man ls
```

Then type:

```
/option
```

Press **Enter**.

To go to the next result, press:

```
n
```

To quit, press:

```
q
```

Search a Man Page With grep

```
man ls | grep "sort"
```

This searches for the word `sort` inside the manual page for `ls`.

9. How to Redirect Output `>`, `>>`, and `|`

Redirection sends the output of a command to a file or to another command.

`>` Redirect and Overwrite

The `>` symbol sends output to a file. If the file already has content, it overwrites it.

Example

```
ls > files.txt
```

This saves the output of `ls` into `files.txt`.

`>>` Append Output

The `>>` symbol adds output to the end of a file.

Example

```
date >> files.txt
```

This adds the current date to the end of `files.txt`.

`|` Pipe

The pipe `|` sends the output of one command to another command.

Example

```
ls | grep ".txt"
```

This lists files and then filters only the files that contain `.txt`.

10. How to Append the Output of a Command to a File

To append output, use `>>`.

Example

```
echo "This is a new line" >> notes.txt
```

This adds the sentence to the end of `notes.txt`.

Another Example

```
ls -l >> notes.txt
```

This adds the long list output of `ls -l` to the end of `notes.txt`.

11. How and When to Redirect Output of a Command to Another Command Pipes

A pipe sends the output of one command to another command.

Syntax

```
command1 | command2
```

Example

```
ls | grep "notes"
```

Explanation

- `ls` lists files.
- `grep "notes"` searches for files that contain the word `notes`.

Another Example

```
cat finalNotes.md | wc -l
```

This counts the lines inside `finalNotes.md`.

When to Use Pipes

Use pipes when you want to:

- Search output.
- Filter output.
- Count lines or words.
- Combine commands.
- Send one command result into another command.

12. How to Use echo and Output Redirection to Create a New File That Contains Text

The `echo` command displays text. With redirection, it can create a file.

Create a New File With Text

```
echo "Hello World" > hello.txt
```

This creates a file named `hello.txt` with the text `Hello World`.

Append More Text

```
echo "This is another line" >> hello.txt
```

This adds another line to the file.

Display the File

```
cat hello.txt
```

13. How to Use Wildcards

Wildcards are symbols used to match file names.

They are useful for copying and moving multiple files at the same time.

Common Wildcards

| Wildcard | Meaning |
|----------|------------------------------------|
| * | Matches anything |
| ? | Matches one character |
| [] | Matches characters inside brackets |

Copy All Text Files

```
cp *.txt backup/
```

This copies all files ending in `.txt` into the `backup` directory.

Move All JPG Files

```
mv *.jpg Pictures/
```

This moves all `.jpg` files into the `Pictures` directory.

Copy Files With One Character Difference

```
cp file?.txt backup/
```

This can match files like:

```
file1.txt  
file2.txt  
fileA.txt
```

Move Files That Start With report

```
mv report* reports/
```

This moves all files that begin with **report**.

14. How to Use Brace Expansion

Brace expansion is used to create many files or directories with one command.

It is useful for creating entire directory structures quickly.

Example 1: Create Multiple Directories

```
mkdir -p school/{math,english,science}
```

Result

```
school/  
├─ math  
├─ english  
└─ science
```

Example 2: Create Wallpapers Directory Structure

```
mkdir -p wallpapers/cars/{1080p,2k,4k}
```

Result

```
wallpapers/  
└─ cars
```

```
├─ 1080p
├─ 2k
└─ 4k
```

Example 3: Create Assets Directory Structure

```
mkdir -p assets/{imgs,video}/{large,small}
```

Result

```
assets/
├─ imgs
│   ├── large
│   └─ small
└─ video
    ├── large
    └─ small
```

15. How to Create a Simple Hello World Shell Script

A shell script is a file that contains commands.

Step 1: Create the File

```
touch hello.sh
```

Step 2: Open the File

```
nano hello.sh
```

Step 3: Write the Script

```
#!/bin/bash
```

```
echo "Hello World"
```

Step 4: Make the Script Executable

```
chmod +x hello.sh
```

Step 5: Run the Script

```
./hello.sh
```

Output

```
Hello World
```

16. How to Use Variables in a Shell Script

Variables store information inside a shell script.

Example 1

```
#!/bin/bash  
  
name="Frandy"  
  
echo "Hello, $name"
```

Output

```
Hello, Frandy
```

Example 2


```
#!/bin/bash

course="CIS106"
assignment="Final Exam Study Notes"

echo "Course: $course"
echo "Assignment: $assignment"
```

Important Rule

Do not put spaces around the equal sign.

Correct

```
name="Frandy"
```

Incorrect

```
name = "Frandy"
```

17. Command Reference

For each command, include:

- **Definition**
- **Syntax or usage**
- **Examples**

a. **awk**

Definition

awk is used to process text and work with columns in files.

Syntax

```
awk 'pattern { action }' file
```

Examples

Example 1: Print the First Column

```
awk '{print $1}' students.txt
```

This prints the first column from `students.txt`.

Example 2: Print the First and Third Columns

```
awk '{print $1, $3}' students.txt
```

This prints the first and third columns.

Example 3: Print Lines That Contain a Word

```
awk '/Linux/ {print}' notes.txt
```

This prints lines that contain the word `Linux`.

Example 4: Print Lines Where the Third Column Is Greater Than 80

```
awk '$3 > 80 {print}' grades.txt
```

This prints lines where the third column is greater than 80.

b. `cat`

Definition

`cat` displays the content of a file.

Syntax

```
cat file
```

Examples

Example 1: Display a File

```
cat notes.txt
```

This shows the content of `notes.txt`.

Example 2: Display Multiple Files

```
cat file1.txt file2.txt
```

This displays both files.

Example 3: Combine Files

```
cat file1.txt file2.txt > combined.txt
```

This combines both files into `combined.txt`.

Example 4: Create a File

```
cat > newfile.txt
```

This allows you to type text into a new file. Press `Ctrl + D` to save.

C. `cp`

Definition

`cp` copies files and directories.

Syntax

```
cp source destination
```

Examples

Example 1: Copy a File

```
cp notes.txt backup.txt
```

This copies `notes.txt` and creates `backup.txt`.

Example 2: Copy a File to a Directory

```
cp notes.txt Documents/
```

This copies `notes.txt` into the `Documents` folder.

Example 3: Copy a Directory

```
cp -r project backup_project
```

This copies the folder and everything inside it.

Example 4: Copy All Text Files

```
cp *.txt backup/
```

This copies all `.txt` files into the `backup` folder.

d. `cut`

Definition

`cut` extracts sections from each line of a file.

Syntax

```
cut option file
```

Examples

Example 1: Cut Characters

```
cut -c 1-5 names.txt
```

This shows characters 1 through 5 from each line.

Example 2: Cut the First Column in a CSV File

```
cut -d "," -f 1 students.csv
```

This shows the first field using comma as the delimiter.

Example 3: Cut the Second Field

```
cut -d ":" -f 2 data.txt
```

This shows the second field using `:` as the delimiter.

e. `grep`

Definition

`grep` searches for text patterns inside files.

Syntax

```
grep "word" file
```

Examples

Example 1: Search for a Word

```
grep "Linux" notes.txt
```

This finds lines that contain `Linux`.

Example 2: Ignore Case

```
grep -i "linux" notes.txt
```

This finds `linux`, `Linux`, and `LINUX`.

Example 3: Show Line Numbers

```
grep -n "error" log.txt
```

This shows matching lines with line numbers.

Example 4: Search Many Files

```
grep "hello" *.txt
```

This searches for `hello` inside all `.txt` files.

f. `head`

Definition

`head` displays the first lines of a file.

Syntax

```
head file
```

Examples

Example 1: Show First 10 Lines

```
head notes.txt
```

This shows the first 10 lines by default.

Example 2: Show First 5 Lines

```
head -n 5 notes.txt
```

This shows the first 5 lines.

Example 3: Show First 20 Lines

```
head -n 20 notes.txt
```

This shows the first 20 lines.

g. `ls`

Definition

`ls` lists files and directories.

Syntax

```
ls [options] [directory]
```

Examples

Example 1: List Files

```
ls
```

This lists files in the current directory.

Example 2: Long List

```
ls -l
```

This lists files with more details.

Example 3: Show Hidden Files

```
ls -a
```

This shows hidden files.

Example 4: Human Readable Sizes

```
ls -lh
```

This shows file sizes in a readable format.

h. `man`

Definition

`man` displays the manual page for a command.

Syntax

```
man command
```

Examples

Example 1: Manual for **ls**

```
man ls
```

This opens the manual page for **ls**.

Example 2: Manual for **grep**

```
man grep
```

This opens the manual page for **grep**.

Example 3: Manual for **cp**

```
man cp
```

This opens the manual page for **cp**.

i. **mkdir**

Definition

mkdir creates directories.

Syntax

```
mkdir directory_name
```

Examples

Example 1: Create One Directory


```
mkdir projects
```

This creates a folder named `projects`.

Example 2: Create Multiple Directories

```
mkdir notes homework labs
```

This creates three directories.

Example 3: Create Parent Directories

```
mkdir -p school/math/algebra
```

This creates the full path if it does not already exist.

Example 4: Use Brace Expansion

```
mkdir -p school/{math,english,science}
```

This creates multiple folders inside `school`.

j. `mv`

Definition

`mv` moves or renames files and directories.

Syntax

```
mv source destination
```

Examples

Example 1: Rename a File

```
mv old.txt new.txt
```

This renames `old.txt` to `new.txt`.

Example 2: Move a File

```
mv notes.txt Documents/
```

This moves `notes.txt` into the `Documents` folder.

Example 3: Move Multiple Files

```
mv *.txt backup/
```

This moves all `.txt` files into the `backup` folder.

Example 4: Rename a Directory

```
mv old_folder new_folder
```

This renames a directory.

k. `tac`

Definition

`tac` displays file content in reverse order, from the last line to the first line.

Syntax

```
tac file
```

Examples

Example 1: Reverse a File

```
tac notes.txt
```

This displays `notes.txt` from bottom to top.

Example 2: Reverse Names

```
tac names.txt
```

This displays the lines in reverse order.

I. `tail`

Definition

`tail` displays the last lines of a file.

Syntax

```
tail file
```

Examples

Example 1: Show Last 10 Lines

```
tail notes.txt
```

This shows the last 10 lines by default.

Example 2: Show Last 5 Lines

```
tail -n 5 notes.txt
```

This shows the last 5 lines.

Example 3: Monitor a File

```
tail -f log.txt
```

This keeps showing new lines added to the file.

m. `touch`

Definition

`touch` creates empty files or updates file timestamps.

Syntax

```
touch file
```

Examples

Example 1: Create a File

```
touch notes.txt
```

This creates an empty file.

Example 2: Create Multiple Files

```
touch file1.txt file2.txt file3.txt
```

This creates three empty files.

Example 3: Create a Markdown File

```
touch finalNotes.md
```

This creates a Markdown file.

Example 4: Create a File With an Absolute Path

```
touch /home/student/Documents/test.txt
```

This creates a file using the full path.

n. tr

Definition

tr translates, replaces, or deletes characters.

Syntax

```
command | tr 'old' 'new'
```

Examples

Example 1: Lowercase to Uppercase

```
echo "hello" | tr 'a-z' 'A-Z'
```

This changes lowercase letters to uppercase.

Example 2: Replace Spaces With Dashes

```
echo "hello world" | tr ' ' '-'
```

This replaces spaces with dashes.

Example 3: Delete Numbers

```
echo "abc123" | tr -d '0-9'
```

This removes numbers.

0. tree

Definition

tree displays files and directories in a tree format.

Syntax

```
tree
```

Examples

Example 1: Display Current Directory

```
tree
```

This shows the current directory as a tree.

Example 2: Display a Specific Directory

```
tree Documents
```

This shows the **Documents** folder as a tree.

Example 3: Show Hidden Files

```
tree -a
```

This shows hidden files in the tree.

Example 4: Show Only Directories

```
tree -d
```

This shows only directories.

Quick Practice Checklist

Use this checklist to make sure you are ready:

- ☐ I can clone a GitHub repository.
 - ☐ I can use **git add**, **git commit**, and **git push**.
 - ☐ I can create a Markdown file with headings, images, lists, and code blocks.
 - ☐ I can convert Markdown to PDF.
 - ☐ I can zip a folder in Debian.
 - ☐ I understand absolute and relative paths.
 - ☐ I can use the **man** command.
 - ☐ I can search inside manual pages.
 - ☐ I can use **>**, **>>**, and **|**.
 - ☐ I can use wildcards.
 - ☐ I can use brace expansion.
 - ☐ I can create a simple shell script.
 - ☐ I can use variables in shell scripts.
 - ☐ I can explain and use all required commands.
-

Source

These notes are based on the topics listed in the CIS106 Final Exam Study Guide:

https://cis106.com/final_exam/final_exam/